

2. Programació amb funcions

En la programació, les funcions són una de les eines més potents i fonamentals. Permeten organitzar el codi en blocs reutilitzables, facilitar la lectura i el manteniment dels programes, i reduir la repetició de codi. En JavaScript, les funcions són ciutadans de primera classe (first-class citizens), cosa que significa que poden ser tractades com qualsevol altra dada: es poden assignar a variables, passar com a paràmetres a altres funcions o fins i tot retornar-se com a resultat.

L'objectiu és entendre no només com es declaren i s'utilitzen les funcions, sinó també com pensar en termes de **modularitat i reutilització** —principis essencials en qualsevol llenguatge de programació modern.

2.1 Funcions predefinides del llenguatge

En tots els llenguatges de programació existeixen **llibreries de funcions** que serveixen per resoldre tasques habituals i repetitives. Aquestes llibreries estalvien als programadors haver d'escriure des de zero funcions comunes que sovint es necessiten en molts programes.

Un llenguatge de programació ben desenvolupat sol incloure **una bona quantitat de funcions i mètodes predefinits**, que faciliten molt la feina. De fet, en alguns casos pot ser més difícil **conèixer totes les llibreries** disponibles que aprendre la sintaxi bàsica del llenguatge.

En el cas de **JavaScript**, disposem de moltes funcions integrades. Com que és un llenguatge orientat a objectes, moltes d'aquestes funcions es troben **dins d'objectes** com Math (per operacions matemàtiques), String (per treballar amb cadenes), Array, Date, etc.

Tot i això, també hi ha **funcions globals** que no pertanyen a cap objecte concret i que podem utilitzar directament, com ara:

- `parseInt()`
- `parseFloat()`
- `isNaN()`
- `eval()`
- `alert()` o `console.log()`

Aquestes funcions **no necessiten conèixer cap objecte previ** i són especialment útils en les primeres etapes d'aprenentatge, ja que ens permeten **interactuar**

amb el llenguatge sense necessitat d'aprendre encara l'estructura completa dels objectes.

En aquesta lliçó ens centrarem justament en aquest tipus de funcions: les que **formen part del nucli de JavaScript** i es poden utilitzar de forma directa.

parseInt()

Converteix una cadena de text (String) en un **nombre enter**. Si la cadena comença amb caràcters no numèrics, retorna NaN.

```
1 parseInt("42");           // 42
2 parseInt("101", 2);       // 5 (base binària)
3 parseInt("3.14");         // 3
4 parseInt("abc");          // NaN
```

parseFloat()

Converteix una cadena de text en un **nombre decimal (coma flotant)**. També pot convertir nombres amb notació exponencial.

```
1 parseFloat("3.1416");     // 3.1416
2 parseFloat("10e2");       // 1000
3 parseFloat("5.5kg");      // 5.5
4 parseFloat("hola");       // NaN
```

isNaN()

Comprova si un valor **no és un número**. Retorna true si el valor és NaN, i false en cas contrari.

```
1 isNaN("hola");           // true (no és un número)
2 isNaN(5);                // false
3 isNaN(NaN);              // true
4 isNaN("123");            // false (es pot convertir a número)
```

`isNaN()` fa una conversió prèvia a nombre. Si es vol evitar això, podem usar `Number.isNaN()` (més estricta).

eval()

Executa codi JavaScript que estigui contingut en una **cadena de text**. És molt potent, però perillós si no es controla bé (pot executar codi maliciós).

```
1 eval("2 + 2");           // 4
2 eval("var x = 5");       // defineix la variable x
3 console.log(x);          // 5
```

console.log()

Mostra informació a la **consola del navegador** o entorn d'execució. És molt útil per depurar (comprovar valors, traçar execució...).

```
1 let nom = "Joan";
2 console.log("Hola, " + nom); // Hola, Joan
```

alert()

Mostra una **finestra emergent (popup)** amb un missatge. S'utilitza per **avisos simples**, però no és recomanable en aplicacions modernes.

```
1 alert("Benvingut al curs de JavaScript!");
```

Quan s'executa, bloqueja l'execució del programa fins que l'usuari tanqui l'avís.

No volem donar una impressió equivocada amb aquesta petita llista de funcions natives de JavaScript. En realitat, existeixen moltes més funcions que anirem veient al llarg d'aquest material.

La diferència és que moltes d'aquestes funcions estan associades a objectes concrets. Per exemple:

- Les funcions per treballar amb **cadenes de caràcters** formen part de l'objecte `String`.
- Les funcions per fer **càlculs matemàtics avançats** estan dins de l'objecte `Math`.
- També hi ha funcions relacionades amb el **document HTML** (`document`) i amb la **finestra del navegador** (`window`).

A mesura que avancem en l'aprenentatge, **anirem descobrint aquestes funcions** i el seu ús pràctic dins del model d'objectes que ofereix JavaScript.

2.2 Funcions definides per l'usuari

Les **funcions en JavaScript** són blocs de codi executables als quals podem **passar paràmetres** i operar amb ells. Ens serveixen per **modularitzar els nostres programes** i estructurar-los en **blocs lògics que realitzin una tasca concreta**.

Això fa que el nostre codi sigui més **ordenat, llegible i fàcil de mantenir**, especialment quan el projecte creix.

2.2.1 Declaració de funció

Les funcions es declaren utilitzant la paraula reservada `function`, seguida habitualment d'un nom identificatiu per poder invocar-les més endavant en el codi.

Normalment, quan una funció finalitza la seva execució, retorna un valor mitjançant la paraula clau `return`. Aquest valor pot ser utilitzat per altres parts del programa.

```
1 function saludar(nom) {  
2   return "Hola, " + nom + "!";  
}
```

```
3 }  
4  
5 saludar("Joan"); // "Hola, Joan!"  
6 saludar("Marta"); // "Hola, Marta!"
```

Si no passem cap paràmetre:

```
1 saludar(); // "Hola, undefined!"
```

2.2.2 Funcions anònimes i assignació a variables

A JavaScript, podem crear **funcions sense nom** i assignar-les a una variable. Aquestes funcions s'anomenen **anònimes**, perquè no tenen identificador propi (no porten nom després de `function`).

Això ens permet **guardar la funció com si fos una dada més**, i després cridar-la utilitzant el nom de la variable.

```
1 // Assignem una funció anònima a la variable 'sumar'  
2 let sumar = function(a, b) {  
3   return a + b;  
4 };  
5  
6 // Ara podem cridar la funció com si fos una funció normal  
7 sumar(3, 4); // Retorna 7
```

2.2.3 Funcions fletxa (Arrow Functions)

Les **funcions fletxa** són una manera **més concisa** i moderna d'escriure funcions en JavaScript. Van ser introduïdes amb **ES6 (ECMAScript 2015)** i són molt utilitzades actualment, especialment en programació funcional i amb llibreries modernes com React o Vue.

Característiques principals:

- **Sintaxi més curta:** no cal escriure la paraula `function`.
- **Retorn implícit:** si el cos de la funció és una sola expressió, el resultat es retorna automàticament sense necessitat de `return`.
- **No tenen `this` propi:** hereten el `this` del context on han estat creades (important en funcions dins de mètodes o *callbacks*).
- No tenen arguments, `super` ni poden ser usades com a constructors (`new`).

Exemple bàsic:

```
1 const multiplicar = (a, b) => a * b;  
2  
3 multiplicar(3, 4); // 12
```

És equivalent a:

```
1 function multiplicar(a, b) {  
2   return a * b;  
3 }  
4  
5 multiplicar(3, 4); // 12
```

Quan només hi ha un paràmetre, podem **ometre els parèntesis**:

```
1 const doble = x => x * 2;
```

Equivalent a:

```
1 function doble(x) {  
2   return x * 2;  
3 }
```

Quan tenim zero, o més de dos paràmetres, cal mantenir els parèntesis:

```
1 const sumar = (a, b, c) => a + b + c;  
2 const mostrar = () => console.log("Hola món!");
```

Si la funció té més d'una línia o es vol fer servir return, cal obrir claus {}:

```
1 const mostrar = nom => {  
2   console.log("Hola " + nom);  
3   return true;  
4 };
```

Què entenem per "JavaScript modern"?

Quan parlem de **JavaScript modern**, ens referim a totes aquelles millores i noves funcionalitats que el llenguatge ha anat incorporant a partir de l'estàndard **ES6 (ECMAScript 2015)** i les versions posteriors. Aquestes novetats han fet que JavaScript sigui més potent, més fàcil d'escriure i més proper als llenguatges de programació actuals.

Alguns exemples d'aquest "JavaScript modern" són:

- **Funcions fletxa (arrow functions)** → sintaxi més curta i clara.
- **let i const** → millor control de l'abast (*scope*) de les variables.
- **Templates literals** → permeten inserir variables dins de cadenes amb `${}`.
- **Classes i mòduls** → per organitzar millor el codi i reutilitzar-lo.
- **Promeses i async/await** → nova manera d'abordar la programació asíncrona.

2.2.4 Paràmetres

Podem assignar valors per defecte en cas que no es passin arguments:

```
1 function saludar(nom = "amic") {  
2   return 'Hola, ${nom}';  
3 }  
4  
5 saludar(); // "Hola, amic"  
6 saludar("Pau"); // "Hola, Pau"
```

Encara que no especifiquem paràmetres, podem accedir als arguments passats amb arguments:

```
1 function llistar() {  
2   for (var i = 0; i < arguments.length; i++) {  
3     console.log(arguments[i]);  
4   }  
5 }  
6  
7 llistar("a", "b", "c");  
8 // Mostra: a, b, c
```

arguments només funciona
amb funcions normals, no
amb funcions fletxa.

2.2.5 Àmbit (Scope) de les variables

L'**àmbit (scope)** determina **on és visible una variable** dins del codi. Un bloc és qualsevol secció de codi delimitada per claus {}, com per exemple dins d'un if, for, while, function, etc. Les variables declarades dins d'una funció només són visibles en aquell entorn (àmbit local). Les de fora són globals.

Quan declarem una variable amb var, és **visible a tota la funció (àmbit global)** on es troba, encara que la definim dins d'un bloc. Això pot provocar errors difícils de detectar i és per això que no es recomana el seu ús.

Exemple:

```
1 function prova() {  
2   if (true) {  
3     var x = 5;  
4   }  
5   console.log(x); // 5 (tot i estar fora del bloc if)  
6 }
```

Les variables declarades amb let o const **només existeixen dins del bloc (àmbit local)** on han estat definides. Això evita errors per redefinicions accidentals o accessos indeguts.

Exemple amb let:

```
1 function prova() {
```

```
2   if (true) {  
3       let x = 5;  
4   }  
5   console.log(x); // Error: x is not defined  
6 }
```

Exemple amb const:

```
1 {  
2     const pi = 3.14;  
3 }  
4  
5 console.log(pi); // Error: pi is not defined
```

2.3 Conceptes avançats de funcions

En aquesta secció veurem com les funcions a JavaScript van molt més enllà de simples blocs de codi. Poden guardar informació del seu context (*closures*), actuar com a objectes de primera classe i fins i tot servir per crear altres funcions o objectes. També entendrem millor el paper del `this`, un concepte clau per treballar amb el context d'execució.

2.3.1 Closures (funcions tancades)

Un **closure** és una funció **que s'emporta amb ella l'entorn on va ser creada**, és a dir, **recorda les variables locals de la seva funció mare**, fins i tot després que aquesta hagi acabat d'executar-se.

Això permet:

- Encapsular dades privades que no es poden accedir directament des de fora.
- Crear funcions personalitzades amb valors fixats (com en configuradors, factories, etc.).

Anem a veure un exemple:

```
1 function crearComptador() {  
2     let comptador = 0;  
3  
4     return function() {  
5         comptador++;  
6         return comptador;  
7     };  
8 }  
9  
10 const cont1 = crearComptador();  
11 console.log(cont1()); // 1
```

```
12 console.log(cont1()); // 2
13
14 const cont2 = crearComptador();
15 console.log(cont2()); // 1
```

Cada `crearComptador()` té la seva pròpia còpia privada de comptador, que no és compartida amb altres instàncies. Fixem-nos que posem `cont1()` i no `cont1` perquè la funció `crearComptador()` retorna una funció.

2.3.2 Funcions com a objectes

A JavaScript, **les funcions són tractades com valors**, igual que els números, cadenes o arrays. Això vol dir que **una funció és un objecte de primera classe** (*first-class object*), cosa que ens permet:

- Assignar funcions a variables
- Passar funcions com a arguments a altres funcions
- Retornar funcions com a resultat
- Guardar funcions dins d'estructures de dades (arrays, objectes...)

Això obre la porta a una programació molt més flexible i potent, com per exemple:

- *Callbacks*
- *Closures*
- Funcions de més alt nivell
- Programació funcional

Què és la programació funcional?

La programació funcional és un estil de programació que tracta les **funcions com a primers ciutadans** (*first-class citizens*). Això vol dir que les funcions no només serveixen per executar codi, sinó que també es poden **guardar en variables, passar com a arguments i retornar com a resultat** d'altres funcions.

Aquest enfocament permet escriure programes més **modulars, reutilitzables i fàcils de provar**, ja que cada funció és independent i no depèn de l'estat global.

Exemples típics en JavaScript:

- Passar una funció a `map()`, `filter()` o `forEach()`.
- Retornar funcions especialitzades mitjançant *closures*.
- Crear arrays d'operacions que podem executar dinàmicament.

Assignar funcions a variables:


```
1 const sumar = function(a, b) {
2   return a + b;
3 };
4
5 console.log(sumar(2, 5)); // 7
6 typeof(sumar); // fuction
```

Funció com a paràmetre:

```
1 function operar(op) {
2   return op(3, 4); // executem la funció que rebem com a parà
      metre
3 }
4
5 // Passem una funció fletxa com a argument
6 operar((a, b) => a * b); // 12
7
8 // També podem utilitzar una funció tradicional
9 operar(function(a, b) {
10   return a * b;
11 });
```

Aquí la funció `operar` rep una altra funció com a paràmetre (`op`) i la crida internament amb dos valors. Aquest patró és molt comú en *callbacks* i *frameworks* moderns.

Retornar funcions:

```
1 function crearOperadorMultiplicacio(x) {
2   return function(y) {
3     return x * y;
4   };
5 }
6
7 const perTres = crearOperadorMultiplicacio(3);
8 console.log(perTres(4)); // 12
```

En aquest cas, estem **retornant una funció que recorda el valor de x** (*closure*), creant una funció especialitzada en multiplicar per 3.

Guardar funcions dins d'un array:

```
1 const operacions = [
2   (a, b) => a + b,
3   (a, b) => a - b,
4   (a, b) => a * b,
5   (a, b) => a / b,
6 ];
7
8 console.log(operacions[2](6, 2)); // 12 (multiplicació)
9 // Accedim al tercer element de l'array
```

Les funcions també tenen propietats

Les funcions són objectes especials i poden tenir **propietats pròpies**:

Callback

Anomenem *callback* a una funció que passem com a paràmetre a una altra funció perquè s'executi més endavant, quan sigui necessari. S'utilitza molt en JavaScript per gestionar accions que triguen temps (com llegir dades, esperar un esdeveniment o treballar amb temporitzadors).

```
1 function saluda(nom) {  
2   return "Hola " + nom;  
3 }  
4  
5 saluda.descripcion = "Funció que saluda";  
6  
7 console.log(saluda.descripcion); // "Funció que saluda"
```

2.3.3 Funcions com a fàbriques (closures + mètodes)

Una **fàbrica de funcions** o **factory function** és una funció que crea i retorna un objecte amb mètodes propis. Aquesta tècnica permet **encapsular dades** mitjançant *closures* i retornar funcions que poden accedir-hi de manera segura.

Aquesta estratègia s'utilitza quan volem:

- Crear **objectes personalitzats** sense fer servir class.
- Mantenir **valors interns privats**.
- Exposar només una **interfície controlada** (funcions públiques).

Anem a veure un exemple on tindrem una variable **privada** (n) que només serà accessible dins de la funció comptador:

```
1 function comptador() {  
2   let n = 0; // variable privada  
3  
4   return {  
5     incrementar: () => ++n, // incrementa i retorna el nou  
6       valor  
7     decrementar: () => --n, // decrementa i retorna el nou  
8       valor  
9     valor: () => n // retorna el valor actual  
10  };  
11 }
```

Cada mètode és una **funció fletxa que forma part d'un closure**, i pot accedir a la variable `n` gràcies al context tancat.

Ús del comptador:

```
1 const c = comptador();  
2  
3 c.incrementar(); // 1  
4 c.incrementar(); // 2  
5 c.valor(); // 2  
6  
7 c.decrementar(); // 1  
8  
9 // La variable n no és accessible des de fora  
10 console.log(c.n); // undefined
```

Això garanteix que **només es pot modificar n a través dels mètodes que hem dissenyat**, una manera molt neta d'encapsular dades.

Els avantatges d'aquest patró de disseny són:

- **Seguretat**: no exposem dades directament, només funcions controlades.
- **Reutilització**: podem crear múltiples instàncies independents.

```
1 const a = comptador();
2 const b = comptador();
3
4 a.incrementar(); // 1
5 b.incrementar(); // 1
6 a.incrementar(); // 2
7 b.valor();       // 1
```

Cada instància té **la seva pròpia còpia privada** de n.

2.3.4 El context "this"

En JavaScript, **this** és una **paraula clau** que fa referència al **context d'execució actual**, és a dir, **l'objecte que *posseix* la funció** quan s'executa.

Però **el valor de this canvia segons com s'invoca la funció**, i això pot ser confús si no ho coneixem bé.

Les funcions fletxa **no tenen el seu propi this**, sinó que prenen el **this** de l'àmbit on han estat creades. Això pot evitar errors en certs contextos com en *callbacks* dins d'objectes:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(() => {
4     console.log("Hola, soc " + this.nom); // correcte: this fa
5     // referència a Persona
6   }, 1000);
7 }
```

En canvi, amb la **function clàssica** no funcionaria igual:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(function() {
4     console.log("Hola, soc " + this.nom); // incorrecte: this no
5     // apunta a Persona
6   }, 1000);
7 }
```

En els exemples anteriors les funcions fletxa ens han anat bé perquè volíem accedir a una propietat que estava al mateix nivell de l'àmbit on havia estat creada.

Però a vegades ens pot jugar males passades. Fixem-nos en aquest exemple:

```
1 const obj = {  
2   nom: "Marc",  
3   normal: function () {  
4     console.log(this.nom); // "Marc"  
5   },  
6   fletxa: () => {  
7     console.log(this.nom); // undefined (this apunta a context  
8       global)  
9   },  
10 };  
11 obj.normal(); // "Marc"  
12 obj.fletxa(); // undefined
```

Quan cridem `obj.normal()`, la funció s'executa **com a mètode de l'objecte `obj`**, i per tant **`this` apunta a `obj`**. Això és el comportament normal que esperem.

Les **funcions fletxa no tenen `this` propi**. Quan les definim, **hereten el `this` del context en què van ser creades**.

La funció fletxa **no hereta el `this` de l'objecte `obj`**, perquè **l'objecte literal no crea un nou àmbit de `this`**. Això vol dir que `this` apunta **al context exterior**, que és el **context global** (o `undefined` en `use strict`).

Per tant, quan fem `obj.fletxa()`, el `this` no és `obj`, sinó el `this` global.

Quan convé fer servir arrow functions?

- Quan necessitem una funció curta i clara.
- Quan volem heretar el `this` del context actual.
- En *callbacks*, com `.map()`, `.filter()`, `.forEach()`, etc. (els veurem més endavant).

Quan no convé usar-les:

- Quan necessitem accedir a `this` d'un objecte creat amb `function` (ex. mètodes)
- Quan volem una funció constructora amb `new` (no és possible)

3. Programació amb arrays i objectes

En aquesta lliçó explorarem com JavaScript permet treballar amb dades estructurades i comportaments associats mitjançant objectes. Aprendre a crear i gestionar **arrays** i **objectes** definits pel programador, i introduïrem els principis bàsics de la programació orientada a objectes (POO), com ara classes, propietats i mètodes. Finalment, veurem alguns exemples pràctics que mostren com aplicar aquests conceptes per escriure codi més modular, reutilitzable i fàcil de mantenir.

3.1 Arrays. Objectes definits pel programador

En aquest apartat veurem com JavaScript ens permet organitzar i gestionar dades de manera eficient mitjançant arrays i objectes. Els arrays són col·leccions ordenades d'elements, ideals per emmagatzemar llistes o conjunts de valors, mentre que els objectes ens permeten agrupar informació relacionada en una mateixa estructura amb claus i valors. A més, combinant aquests elements, podem crear estructures de dades complexes adaptades a les necessitats del nostre programa.

3.1.1 Arrays com estructures de dades

Un **array** és una estructura que ens permet **emmagatzemar diversos valors en una sola variable**, de forma **ordenada**. És molt útil quan volem treballar amb **llistes de dades**, com noms, números, objectes, etc.

Els arrays en JavaScript poden contenir **qualsevol mena de dada**, fins i tot barrejats (nombres, *strings*, booleans, objectes, altres arrays...).

Creació d'un array

A continuació, veurem alguns exemples de com crear arrays.

```
1 let buida = []; // Array buit
2 let numeros = [1, 2, 3, 4]; // Array numèric
3 let noms = ["Anna", "Pau", "Joan"]; // Array d'Strings
4 let mesclat = [1, "hola", true, [10, 20]]; // Array mixt
```

També es pot fer amb el constructor:

```
1 let colors = new Array("vermell", "blau", "verd");
```

Tot i això, la forma recomanada és la de claudàtors ([]), per ser més clara i segura.

Accés als elements

Els elements es compten des de l'índex zero (0 és el primer element).

```
1 let numeros = [10, 20, 30];
2 console.log(numeros[0]); // 10
3 console.log(numeros[2]); // 30
```

Longitud de l'array

Cada array té una propietat `.length` que indica quants elements conté:

```
1 numeros.length; // 3
```

Aquesta propietat també es pot usar per recórrer l'array amb un bucle:

```
1 for (let i = 0; i < numeros.length; i++) {
2   console.log(numeros[i]);
3 }
```

Mètodes útils dels arrays

Els arrays disposen de molts **mètodes integrats** per facilitar les operacions habituals. A continuació veurem un llistat dels més usuals:

TAULA 3.1. Mètodes útils dels arrays

Mètode	Què fa
<code>push()</code>	Afegeix un element al final
<code>pop()</code>	Elimina i retorna l'últim element
<code>unshift()</code>	Afegeix un element al principi
<code>shift()</code>	Elimina i retorna el primer element
<code>join(separador)</code>	Uneix tots els elements en un <i>string</i> , separats per separador. El separador per defecte és la coma ,
<code>concat()</code>	Uneix dos arrays en un de sol
<code>slice(inici, fi)</code>	Retorna una part de l'array (no modifica l'original)
<code>splice()</code>	Afegeix/elimina elements de l'array (sí que modifica l'original)
<code>indexOf(elem)</code>	Retorna la posició d'un element (o -1 si no hi és)
<code>includes(elem)</code>	Indica si un element es troba dins l'array

Anem a veure alguns exemples del mètodes anteriors:

```
1 let fruites = ["poma", "plàtan", "kiwi"];
2
3 fruites.push("maduixa");           // ["poma", "plàtan", "kiwi", "
   maduixa"]
```

```
4 fruites.shift();           // ["plàtan", "kiwi", "maduixa"]
5 fruites.includes("kiwi");   // true
6 fruites.indexOf("plàtan");  // 0 (Ha canviat després d'
    aplicar el mètode shift)
```

Iterar amb ".forEach()"

El mètode `.forEach()` ens permet **recórrer tots els elements d'un array** i executar una funció per a cadascun d'ells. És una alternativa moderna i més llegible que els bucles `for`.

No retorna cap nou array: simplement **executa una acció per cada element**.

```
1 let animals = ["gat", "gos", "ocell"];
2
3 animals.forEach(function(animal, index) {
4     console.log(index + ": " + animal);
5 });
```

També es pot fer amb funcions fletxa:

```
1 animals.forEach((animal, i) => console.log(i + ": " + animal));
```

Transformar arrays: ".map()" i ".filter()"

Els mètodes `.map()` i `.filter()` ens permeten crear un nou array a partir de l'original:

- `map()` transforma cada element i retorna un array nou amb els resultats.
- `filter()` selecciona només els elements que compleixen una condició.

Aquests mètodes no modifiquen l'array original.

```
1 let nums = [1, 2, 3, 4];
2
3 // Multiplica per 2 cada element
4 let dobles = nums.map(n => n * 2); // [2, 4, 6, 8]
5
6 // Filtra els nombres parells
7 let parells = nums.filter(n => n % 2 === 0); // [2, 4]
```

3.1.2 Objectes com estructures de dades

A JavaScript, un **objecte** és una estructura que ens permet **emmagatzemar dades en forma de clau-valor**. És una eina fonamental per representar entitats més complexes i flexibles que un array, com ara una persona, un cotxe, un usuari, etc.

Mentre que els arrays organitzen les dades per **posició numèrica (índexs)**, els objectes ho fan per **claus amb nom (strings)**.

Creació d'un objecte

A continuació, es mostra com crear un **objecte literal**. Un objecte literal en JavaScript és una forma directa i senzilla de crear un objecte amb *claus* (propietats) i *valors*.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true  
5 };
```

Cada propietat té una *clau* (ex. “nom”) i un *valor* associat (ex. “Anna”).

Accés a propietats

Podem accedir a les propietats amb notació de punt o notació de claudàtors:

```
1 // Punt (dot notation)  
2 console.log(persona.nom); // "Anna"  
3  
4 // Claudàtors (bracket notation)  
5 console.log(persona["edat"]); // 28
```

Notació amb claudàtors

La notació amb claudàtors és útil quan la clau és una variable o conté espais:

```
1 let clau = "viu";  
2 console.log(persona[clau]); // true
```

Modificar i afegir propietats

Els objectes en JavaScript són **dinàmics**, la qual cosa vol dir que podem afegir noves propietats o modificar-ne les existents en qualsevol moment, fins i tot després d'haver estat creats.

Aquesta flexibilitat fa que els objectes siguin molt útils com a contenidors d'informació estructurada.

```
1 persona.edat = 29; // Modifiquem  
2 persona.professio = "dissenyadora"; // Afegim nova propietat
```

Mètodes dins d'un objecte literal

A més de propietats amb valors com números o textos, un objecte literal també pot incloure **funcions**. Quan una funció forma part d'un objecte, l'anomenem **mètode**.

Aquests mètodes permeten definir accions o comportaments relacionats amb l'objecte.

```
1 let persona = {
2   nom: "Anna",
3   edat: 28,
4   viu: true,
5   saludar() {
6     console.log('Hola! Soc ${this.nom}.');
7   }
8 };
9
10 persona.saludar(); // "Hola! Soc Anna."
```

Els mètodes poden accedir a les propietats de l'objecte mitjançant la paraula clau `this`, que fa referència a l'objecte mateix.

Eliminar propietats

La paraula clau `delete` serveix per **eliminar propietats d'un objecte**. Quan s'utilitza, **esborra la clau i el valor** associat, de manera que ja no existeixen dins de l'objecte.

```
1 delete persona.viu;
```

També podem eliminar **mètodes** (funcions dins d'un objecte), ja que internament són propietats com qualsevol altra.

```
1 let persona = {
2   nom: "Anna",
3   edat: 29,
4   saludar() {
5     console.log('Hola, soc ${this.nom}');
6   }
7 };
8
9 persona.saludar(); // "Hola, soc Anna"
10 delete persona.saludar; // Eliminem el mètode
11 console.log(persona.saludar); // undefined
```

Després d'eliminar-lo, el mètode ja no està disponible dins de l'objecte.

Recórrer les propietats amb "for...in"

Quan treballem amb objectes, sovint volem **recórrer totes les seves propietats** per llegir o processar els valors que contenen. En JavaScript, això es pot fer fàcilment amb el bucle especial **for...in**.

Aquest bucle ens permet **iterar per totes les claus (propietats enumerables)** d'un objecte.

```
1 for (let clau in persona) {
2   console.log(clau + ": " + persona[clau]);
}
```

```
3 }  
4 // nom: Anna  
5 // edat: 29  
6 // professio: dissenyadora
```

Els objectes són especialment útils quan necessitem **representar entitats complexes** que tenen diverses característiques o propietats. Per exemple, si volem descriure un usuari, un llibre o un producte, cadascun d'aquests elements tindrà múltiples dades associades, com el nom, l'edat, el preu o la categoria. En aquests casos, un array no seria suficient, ja que només ens permet emmagatzemar valors de manera ordenada, però **sense significat semàntic clar**.

A diferència dels arrays, que organitzen la informació per posició (índex), els objectes ens permeten accedir a cada dada mitjançant **una clau descriptiva**, com nom, edat o preu. Això fa que el codi sigui més **llegible i entenedor**, sobretot quan treballem amb dades estructurades.

En resum, utilitzarem objectes sempre que vulguem **estructurar informació amb sentit** (estructures de dades), i no només llistes de valors. Són una eina fonamental per construir programes clars, escalables i fàcils de mantenir.

3.1.3 Estructures de dades complexes

Quan treballem amb dades reals en una aplicació, sovint no n'hi ha prou amb estructures simples com arrays o objectes independents. En aquests casos, JavaScript ens permet **combinar arrays i objectes** per construir **estructures de dades complexes**, molt més potents i representatives del món real.

Arrays que contenen objectes

Una de les combinacions més habituals és tenir un **array d'objectes**. Aquesta estructura és ideal quan volem representar una col·lecció d'elements que comparteixen una mateixa estructura, com ara una llista de llibres, d'usuaris o de productes. Cada objecte de l'array representa un element amb les seves pròpies propietats.

Per exemple, imaginem que volem representar un catàleg de llibres:

```
1 let llibres = [  
2   { titol: "1984", autor: "George Orwell", any: 1949 },  
3   { titol: "Crim i càstig", autor: "Fiódor Dostoievski", any:  
4     1866 },  
5   { titol: "L'ombra del vent", autor: "Carlos Ruiz Zafón", any:  
     2001 }  
];
```

En aquest cas, l'array `llibres` conté tres objectes, cadascun amb informació rellevant sobre un llibre. Això ens permet, per exemple, recórrer tota la llista

amb un bucle i mostrar només els títols o buscar l'autor d'un llibre concret.

Aquesta estructura és especialment útil quan gestionem dades provinents d'una API o una base de dades, ja que aquestes normalment retornen col·leccions d'elements representats com objectes dins d'un array.

Objectes amb propietats que són arrays

També és molt comú el cas invers: **objectes que contenen arrays com a valors d'algunes propietats**. Això ens permet representar característiques que poden tenir **múltiples valors associats**, com per exemple les aficions d'una persona, les notes d'un alumne, o les etiquetes d'un producte.

Observem aquest exemple:

```
1 let usuari = {  
2   nom: "Marta",  
3   edat: 22,  
4   aficions: ["lectura", "escalada", "viatjar"]  
5 };
```

Aquí l'objecte usuari conté una propietat aficions que és un array. Això ens permet treballar de manera eficient amb múltiples valors relacionats amb un sol camp, mantenint una estructura lògica i clara.

Podem accedir-hi i treballar-hi fàcilment:

```
1 console.log(usuari.aficions[1]); // "escalada"  
2 usuari.aficions.push("cinema");
```

Aquest tipus de combinacions ens permeten **modelar la realitat** dins del nostre programa. A diferència d'un full de càlcul o una simple llista de valors, aquí podem representar relacions complexes, jerarquies i col·leccions d'informació estructurada.

És gràcies a aquestes estructures que podem construir funcionalitats com:

- Llistes de tasques amb subtasques
- Perfils d'usuari amb múltiples preferències
- Carretons de compra amb productes i quantitats
- Aplicacions de calendari amb esdeveniments per dies

Dominar les estructures de dades complexes ens permet construir aplicacions **més potents i realistes**. JavaScript ens dona la flexibilitat de **combinar arrays i objectes lliurement**, cosa que és fonamental per treballar amb dades de forma organitzada, neta i escalable.

3.2 Programació orientada a objectes

JavaScript és un llenguatge molt flexible i dinàmic, i una de les seves característiques més potents és que permet programar seguint el paradigma orientat a objectes. Això vol dir que podem organitzar el nostre codi en estructures que encapsulen dades i comportaments, facilitant la reutilització, la modularitat i la comprensió del programa.

Tot i que internament JavaScript es basa en un sistema d'**herència per prototips**, des de la versió **ES6 (2015)** s'introdueixen les **classes**, que ens permeten escriure codi amb una sintaxi més familiar i clara, especialment per a persones acostumades a llenguatges com Java, Python o C++.

La programació orientada a objectes amb JavaScript ens permet **modelar el món real** a través de classes, objectes, propietats i mètodes. Gràcies a la sintaxi moderna de classes, escriure codi estructurat i reutilitzable és molt més senzill i intuïtiu.

Aquest estil de programació és fonamental quan treballem en projectes grans, interfícies complexes o aplicacions que gestionen molts tipus d'entitats i relacions. Aprendre a utilitzar correctament les classes, l'herència i l'encapsulació ens dona una eina potent per escriure **codi net, mantenible i escalable**.

3.2.1 Introducció a les classes

Una classe és un esquema o plantilla que defineix com han de ser els objectes que es creïn a partir d'ella. Ens permet agrupar les propietats i els mètodes que definiran el comportament d'una entitat concreta, com ara un usuari, una mascota, una factura o qualsevol element que volem modelar.

En JavaScript, podem definir una classe utilitzant la paraula clau `class`, i afegir-hi un constructor per inicialitzar les propietats. A diferència d'altres llenguatges, no cal declarar els atributs fora del constructor.

Per exemple, si volem definir una classe `Persona`, podríem escriure el següent:

```
1 class Persona {  
2   constructor(nom, edat) {  
3     this.nom = nom;  
4     this.edat = edat;  
5   }  
6  
7   saludar() {  
8     console.log(`Hola, soc ${this.nom} i tinc ${this.edat} anys`  
9       .');  
10  }  
11 }
```

Quan fem servir el mot clau `new`, estem creant una nova instància d'aquesta classe. Això vol dir que podem crear tantes persones com vulguem, totes amb el seu nom i edat particulars, però compartint el comportament comú definit dins la classe.

```
1 const p1 = new Persona("Anna", 30);
2 const p2 = new Persona("Joan", 22);
3
4 p1.saludar(); // Hola, soc Anna i tinc 30 anys.
5 p2.saludar(); // Hola, soc Joan i tinc 22 anys.
```

3.2.2 El paper de 'this' dins de les classes

Dins d'una classe, la paraula clau `this` fa referència a la instància concreta de l'objecte. És a dir, quan escrivim `this.nom`, estem accedint a la propietat `nom` de l'objecte que s'utilitza en aquell moment. Aquest comportament és essencial per distingir entre els valors que rep el constructor i els que es guarden dins l'objecte.

3.2.3 Encapsulació i modificació de valors

A diferència de llenguatges fortament orientats a objectes, JavaScript no obliga a declarar les propietats com a públiques o privades. Però a partir d'ES2022 ja és possible usar el símbol `#` per definir **propietats privades**, que només poden ser accedides des de dins de la classe.

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   ingressar(quantitat) {
10    this.#saldo += quantitat;
11  }
12
13  veureSaldo() {
14    return this.#saldo;
15  }
16 }
17
18 const c = new Compte("Laura");
19 c.ingressar(100);
20 console.log(c.veureSaldo()); // 100
21 console.log(c.#saldo);       // Error: propietat privada
```

Aquest sistema ens ajuda a protegir les dades internes i evitar que es modifiquin des de fora de la classe, millorant la seguretat i robustesa del codi.

Propietats públiques vs. privades

JavaScript **obliga** a declarar les propietats privades amb # **a dalt de tot dins de la classe**, abans de fer-les servir dins del constructor o de qualsevol mètode. Això forma part de l'especificació del llenguatge.

Per contra, les propietats **públiques** (com `this.titular`) **no cal declarar-les prèviament**. El fet de fer `this.titular = ...` dins del constructor **ja crea automàticament la propietat a l'objecte**.

Ara bé, sovint volem **controlar encara més l'accés o la modificació de certs valors**, però sense haver d'utilitzar mètodes com `veureSaldo()` o `modificarSaldo()`, que poden resultar menys intuïtius. En aquests casos, JavaScript ens ofereix una eina molt útil: els **accessors de dades** o **data accessors**, que s'implementen mitjançant les paraules clau `get` i `set`.

Els accessors ens permeten definir **com es llegeix (get) o com s'escriu (set) una propietat**, però des del punt de vista extern, l'ús serà igual que si accedíssim directament a una propietat. Això fa que el codi sigui més net i expressiu, i alhora ens permet **validar o transformar les dades en segon pla**.

Per exemple, podem reescriure el cas anterior del `Compte` per fer servir `get` i `set` en lloc de mètodes tradicionals:

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   get saldo() {
10    return this.#saldo.toFixed(2) + " €";
11  }
12
13  set saldo(quantitat) {
14    if (quantitat >= 0) {
15      this.#saldo = quantitat;
16    } else {
17      console.log("No es pot assignar un saldo negatiu.");
18    }
19  }
20 }
```

Ara, l'ús extern és molt més natural:

```
1 const compte = new Compte("Laura");
2
3 compte.saldo = 200;           // assignació controlada amb el
   set
4 console.log(compte.saldo);    // lectura controlada amb el get
   → "200.00 €"
5
6 compte.saldo = -50;          // no es permet, salta l'avís
```

Amb aquesta estructura, mantenim la protecció de la propietat interna `#saldo`, però oferim **una interfície d'accés elegant i segura**, molt semblant a les propietats públiques d'altres llenguatges orientats a objectes.

Els *data accessors* són especialment útils quan volem que **el comportament d'una propietat sembli directe, però internament amagui lògica de control o format**, i per tant constitueixen una extensió molt natural de l'encapsulació moderna en JavaScript.

3.2.4 Abstracció

L'**abstracció** és un dels pilars fonamentals de la programació orientada a objectes. Consisteix a **ocultar els detalls interns d'una implementació i mostrar només la interfície necessària** per a qui l'utilitza. L'objectiu és **simplificar la complexitat**, evitant que el codi que utilitza un objecte hagi de conèixer com funciona per dins.

En llenguatges com **Java**, l'abstracció es formalitza mitjançant **classes abstractes i interfaces**, que permeten definir què pot fer un objecte sense dir exactament com ho fa. Això facilita la separació entre definició i implementació.

En canvi, JavaScript no té classes abstractes com a construcció formal, però això no vol dir que no pugui implementar l'abstracció. En JavaScript, podem aconseguir l'abstracció encapsulant detalls interns, fent servir propietats privades (amb #) i exposant només mètodes públics per interactuar amb l'objecte.

Exemple d'abstracció en JavaScript:

```
1 class Sensor {
2   #valorIntern;
3
4   constructor() {
5     this.#valorIntern = Math.random() * 100;
6   }
7
8   obtenirLectura() {
9     return this.#valorIntern.toFixed(2) + " u.";
10  }
11 }
```

En aquest cas, el codi que utilitza el sensor només pot fer:

```
1 const s = new Sensor();
2 console.log(s.obtenirLectura()); // "47.39 u." (per exemple)
```

No pot accedir a #valorIntern ni modificar-lo directament, perquè forma part de la implementació interna. L'usuari només veu el mètode públic obtenirLectura(), que és tot el que necessita per fer servir correctament l'objecte.

Comparació amb un exemple en Java:

```
1 public abstract class Sensor {
2   protected double valorIntern;
3
4   public Sensor() {
5     this.valorIntern = Math.random() * 100;
6   }
7 }
```

```
7
8     public abstract String obtenirLectura();
9 }
10
11 public class SensorTemperatura extends Sensor {
12     @Override
13     public String obtenirLectura() {
14         return String.format("%.2f ºC", valorIntern);
15     }
16 }
```

I el seu ús seria:

```
1 Sensor s = new SensorTemperatura();
2 System.out.println(s.obtenirLectura());
```

En aquest cas, la **classe abstracta Sensor** defineix **què ha de fer** qualsevol sensor (`obtenirLectura()`), però no com. La classe *filla* (`SensorTemperatura`) implementa el comportament concret. Això és **abstracció formal**, amb suport explícit del llenguatge.

3.2.5 Herència entre classes

La programació orientada a objectes també permet **heretar comportaments** d'una classe base. Això vol dir que podem definir una classe general i després crear altres classes més específiques que parteixen de la primera i afegeixen funcionalitats pròpies.

Per fer-ho, JavaScript ens ofereix les paraules clau `extends` i `super`:

```
1 class Animal {
2     constructor(nom) {
3         this.nom = nom;
4     }
5
6     parlar() {
7         console.log(`${this.nom} fa un so.`);
8     }
9 }
10
11 class Gos extends Animal {
12     parlar() {
13         console.log(`${this.nom} borda.`);
14     }
15 }
16
17 const layca = new Animal("Layca");
18 layca.parlar(); // Layca fa un so.
19 const rex = new Gos("Rex");
20 rex.parlar(); // Rex borda.
```

Quan una classe hereta d'una altra, pot **sobreescriure mètodes** per adaptar-ne el comportament, com hem vist amb el mètode `parlar()`. Aquesta sobreescritura de mètodes és coneix com a **polimorfisme**.

3.2.6 Polimorfisme

El **polimorfisme** ens permet definir mètodes amb el mateix nom però comportaments diferents, segons el tipus d'objecte que els implementa. En JavaScript, això es pot aconseguir mitjançant sobreescritura de mètodes en classes filles.

```
1 class Empleat {
2   saludar() {
3     console.log("Hola, soc un empleat.");
4   }
5 }
6
7 class Cap extends Empleat {
8   saludar() {
9     console.log("Hola, soc el cap!");
10  }
11 }
12
13 const e = new Empleat();
14 const c = new Cap();
15
16 e.saludar(); // "Hola, soc un empleat."
17 c.saludar(); // "Hola, soc el cap!"
```

Tots dos objectes tenen el mètode `saludar()`, però el comportament és diferent: això és **polimorfisme**.

3.2.7 Una menció als prototips

Tot i que actualment la majoria de programadors fan servir la sintaxi de classes, és important saber que **les classes són només una forma més clara de treballar amb els prototips**, que són el veritable sistema d'herència de JavaScript. Cada objecte en JavaScript té un enllaç intern al seu prototip, i si no troba una propietat o mètode en ell mateix, la busca al prototip del qual deriva.

Aquesta cadena d'herència es coneix com a **cadena de prototips**. Tot i això, gràcies a les classes, ja no cal definir manualment mètodes al prototip, cosa que abans es feia així:

```
1 function Persona(nom) {
2   this.nom = nom;
3 }
4
5 Persona.prototype.saludar = function() {
6   console.log('Hola, soc ${this.nom}');
7 };
```

Aquest codi funciona, però és molt menys llegible que la versió moderna amb `class`.